Music Playlist Manager – Architettura Software

Software Architecture Design 2026

Corso di Laurea Magistrale in Ingegneria Informatica
Anno Accademico 2025/2026



Docenti: Ritrovato Pierluigi, Apicella Andrea

GitHub: `https://github.com/fra343/Gruppo2-SAD`

Trello: `https://trello.com/b/Mpb9bjYE/sad`

Gruppo 02:

Apicella Francesco	IE22700106
Campagna Lorenzo	IE22700105
Cutolo Raffaele	IE22700113
Gioia Errico	IE22700112

Contents

1 Introduzione **2**
1.1 Obiettivi del prodotto . . . 2
1.2 Approccio adottato . . . 2
1.3 Scelte tecnologiche . . . 2

2 Architettura software **3**
2.1 Pattern architetturale: MVC . . . 3
2.2 Struttura dei package . . . 4
2.3 Design pattern adottati . . . 4
2.3.1 Strategy — Modalità di riproduzione . . . 5
2.3.2 Observer — Aggiornamento UI in tempo reale . . . 5
2.3.3 Command — Supporto per l'Undo . . . 5
2.3.4 Factory — Creazione delle strategie . . . 5
2.3.5 DAO (Data Access Object) — Disaccoppiamento della persistenza . . . 5
2.4 Modello di threading . . . 6
2.5 Diagramma delle classi . . . 6
2.5.1 Lettura del diagramma . . . 6

3 Flusso del sistema **8**
3.1 Avvio dell'applicazione . . . 8
3.2 Aggiunta di una traccia alla libreria . . . 8
3.3 Riproduzione di una playlist . . . 8
3.4 Cambio di modalità di riproduzione a runtime . . . 8
3.5 Modifica di una playlist durante la riproduzione . . . 9
3.6 Annullamento di un'operazione (undo) . . . 9

4 Definition of Done **10**
4.1 Criteri tecnici sul codice . . . 10
4.2 Criteri di testabilità e qualità . . . 10
4.3 Criteri di processo . . . 10

1 Introduzione

Il presente documento descrive il progetto *Music Playlist Manager*, un'applicazione desktop sviluppata in Java. L'applicazione consente all'utente di gestire una libreria di tracce musicali, organizzarle in playlist e simularne la riproduzione attraverso diverse modalità (sequenziale, shuffle e loop), con aggiornamento in tempo reale dell'interfaccia utente.

1.1 Obiettivi del prodotto

L'applicazione si propone di realizzare le seguenti funzionalità principali:

- Gestione (creazione, modifica, eliminazione, visualizzazione) di tracce musicali, ciascuna caratterizzata da titolo, autore, durata, genere e anno di pubblicazione;
- Creazione/eliminazione e gestione di playlist identificate da un nome, con possibilità di aggiungere o rimuovere tracce in qualsiasi momento, anche durante la riproduzione;
- Riproduzione simulata di tracce e playlist con tre modalità intercambiabili (sequenziale, shuffle, loop);
- Aggiornamento in tempo reale dell'interfaccia utente per riflettere lo stato della riproduzione e le modifiche al contenuto delle playlist.

1.2 Approccio adottato

Lo sviluppo segue il processo Scrum, con un'organizzazione in tre sprint che coprono complessivamente il periodo dal 28 maggio al 18 giugno 2026. Coerentemente con il principio di *time-boxing* indicato dalla docenza, il team ha deliberatamente scelto di limitare lo scope del progetto a un sottoinsieme delle user stories complessive, privilegiando la qualità del design, la robustezza del codice e l'aderenza al processo rispetto al numero di funzionalità implementate. Le user stories di priorità medio-bassa sono state mantenute nel Product Backlog ma posizionate come obiettivi opzionali, da affrontare solo dopo aver consolidato il nucleo dell'applicazione.

1.3 Scelte tecnologiche

Linguaggio: Java 17

Build: Maven

UI: JavaFX

Test: JUnit 5

Persistenza: JSON

Versioning: GitHub

IDE: Visual Studio Code

2 Architettura software

Questa sezione descrive l'architettura dell'applicazione Music Playlist Manager, illustrando il pattern architetturale adottato, l'organizzazione in package e i design pattern impiegati per lo sviluppo dell'applicazione.
Le scelte architetturali sono state guidate da tre principi:

- **Alta coesione;**
- **Basso accoppiamento;**
- **Testabilità.**

La trattazione procede in modo incrementale: viene presentato dapprima il pattern architetturale complessivo (sez. 2.1), quindi la sua materializzazione nei package del codice (sez. 2.2) e i design pattern adottati per affrontare problemi specifici (sez. 2.3). Seguono una descrizione del modello di gestione della concorrenza (sez. 2.4) e una lettura guidata del diagramma delle classi (sez. 2.5). La sezione 3 descrive infine il comportamento a runtime attraverso scenari rappresentativi, in modo da mostrare in che modo gli strati architetturali interagiscono nei casi d'uso più significativi.

2.1 Pattern architetturale: MVC

L'applicazione adotta una variante del pattern Model-View-Controller (MVC), aggiungendo uno strato intermedio di ***Service*** che incapsula la logica di business. Questa scelta è naturalmente supportata da JavaFX, che fornisce un meccanismo nativo (FXML) per separare la definizione delle viste dai controller.
I quattro strati dell'architettura sono i seguenti:

- **Model**: contiene le entità di dominio (Track, Playlist, Library) e ne definisce stato e comportamento.
- **View**: definita tramite file FXML, descrive la struttura statica dell'interfaccia utente senza contenere logica applicativa.
- **Controller**: gestisce gli eventi dell'interfaccia utente (click, input, selezioni) e li traduce in invocazioni dei servizi.
- **Service**: incapsula la logica applicativa (creazione di tracce, gestione delle playlist, riproduzione). Dipende dal Model ma è indipendente dalla UI, e può quindi essere testato senza istanziare componenti JavaFX.

2.2 Struttura dei package

L'organizzazione dei package mostra la separazione delle responsabilità descritta sopra.

Package	Descrizione
`Model`	Logica di stato interno alle entità.
`Service`	Espone le operazioni utilizzate dai controller.
`Controller`	Gestione degli eventi UI e delega ai service.
`View`	File FXML, fogli di stile CSS e risorse grafiche (icone).
`Persistence`	Implementazione del salvataggio/caricamento.
`Playback`	Implementazione delle funzionalità di riproduzione.
`Command`	Pattern Command per il supporto all'undo.

Table 1: Organizzazione dei package dell'applicazione.

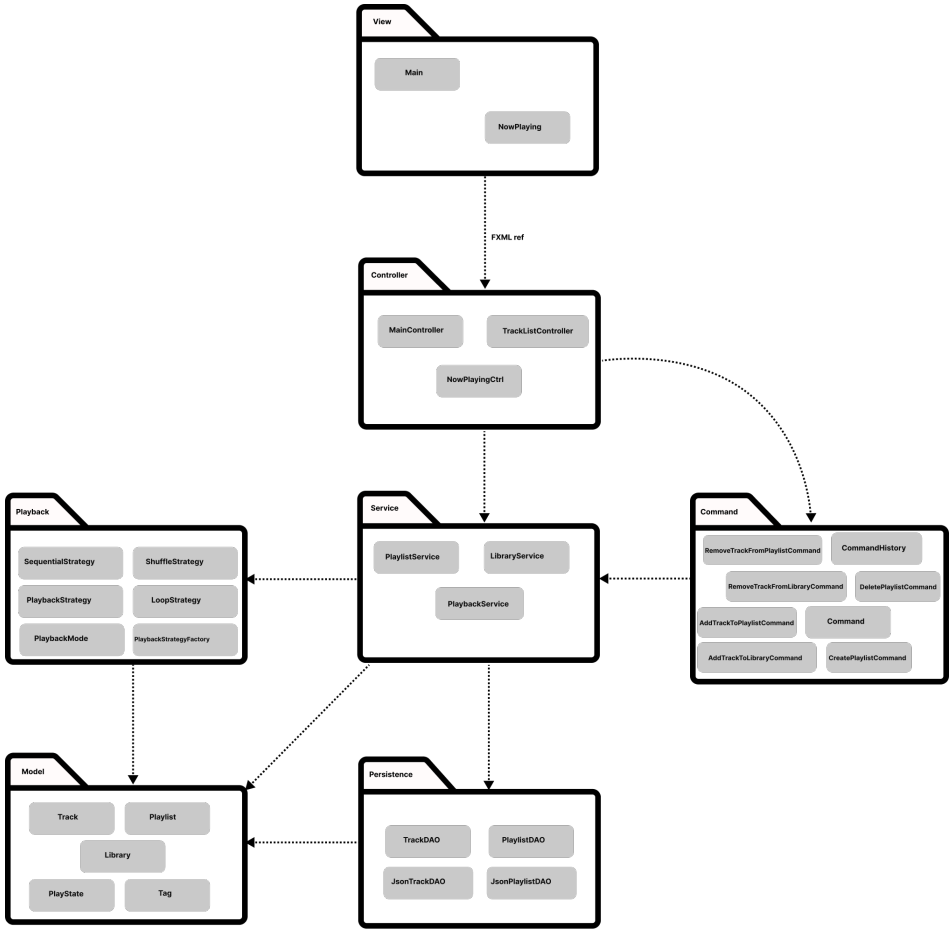


Figure 1: Diagramma dei Package

2.3 Design pattern adottati

L'architettura adotta diversi design pattern per affrontare specifici problemi ricorrenti.

2.3.1 Strategy — Modalità di riproduzione

Il sistema deve supportare tre modalità di riproduzione (sequenziale, shuffle, loop). È necessario poter cambiare modalità a runtime, anche durante la riproduzione, e prevedere l'estensione futura ad altre modalità.
Si definisce un'interfaccia `PlaybackStrategy` con un metodo principale `next(Playlist current, Track currentTrack)` che restituisce la prossima traccia da riprodurre. Tre implementazioni concrete (`SequentialStrategy`, `ShuffleStrategy`, `LoopStrategy`) incapsulano gli algoritmi specifici. Il `PlaybackService` detiene un riferimento alla strategia corrente e la invoca senza conoscerne il tipo concreto.
Il principale vantaggio dell'utilizzo di questo pattern è che il cambio di modalità si riflette in una semplice sostituzione di oggetto, senza alcuna modifica al codice. L'aggiunta di una nuova modalità richiederebbe solo una nuova classe senza toccare nient'altro.

2.3.2 Observer — Aggiornamento UI in tempo reale

L'interfaccia utente deve riflettere in tempo reale lo stato della riproduzione (traccia corrente, tempo trascorso, modalità attiva) e le modifiche al contenuto delle playlist.
L'implementazione si appoggia direttamente al supporto nativo di JavaFX: le collezioni di tracce in `Playlist` e `Library` sono `ObservableList<Track>`, mentre i campi mutabili del `PlaybackService` (traccia corrente, tempo trascorso, stato di riproduzione) sono esposti come property tipizzate (`ObjectProperty<Track>`, `IntegerProperty`, `ObjectProperty<PlayState>`). I controller della UI bindano i componenti grafici (`Label`, `ProgressBar`, `ListView`) a queste property: ogni cambiamento di stato si propaga automaticamente alla view senza callback manuali.
Questa scelta evita di duplicare un meccanismo Observer "casalingo" su un framework che lo offre già, e mantiene tutte le notifiche nello stesso modello di sincronizzazione gestito da JavaFX.

2.3.3 Command — Supporto per l'Undo

Le operazioni modificanti vengono incapsulate in oggetti `Command` con metodi `execute()` e `undo()`. Una classe `CommandHistory` mantiene uno stack delle operazioni eseguite. Anche quando l'undo non sarà ancora disponibile per l'utente, le operazioni passeranno comunque per il meccanismo dei Command, in modo che la funzionalità possa essere abilitata in un secondo momento con minimo impatto. Pensare in termini di Command fin dall'inizio rende l'aggiunta dell'undo un'operazione non invasiva. Un design alternativo (modifica diretta dello stato) renderebbe l'aggiunta dell'undo un refactoring esteso a tutto il codice.

2.3.4 Factory — Creazione delle strategie

I controller della UI non devono dipendere dalle classi concrete delle strategie di riproduzione. Si introduce quindi una classe `PlaybackStrategyFactory` che espone un unico metodo `create(PlaybackMode mode)`, dove `PlaybackMode` è un'enum con valori `SEQUENTIAL`, `SHUFFLE`, `LOOP`. Il metodo restituisce l'istanza appropriata di `PlaybackStrategy` in base al valore ricevuto.
Rispetto a un'esposizione di tre metodi separati (`forSequential()`, `forShuffle()`, `forLoop()`), un'unica firma parametrica centralizza il punto di estensione: aggiungere una nuova modalità richiede un nuovo valore nell'enum, una nuova classe Strategy e un solo ramo nella factory, senza intervenire sui siti di chiamata.

2.3.5 DAO (Data Access Object) — Disaccoppiamento della persistenza

L'applicazione deve salvare e ricaricare tracce e playlist su disco, ma il resto dell'applicazione non deve dipendere dal formato specifico di memorizzazione scelto.
Si definiscono due interfacce, `TrackDAO` e `PlaylistDAO`, che espongono i metodi `save()` e `load()`. Le implementazioni concrete `JsonTrackDAO` e `JsonPlaylistDAO` incapsulano la logica di serializzazione e deserializzazione in formato JSON. Il layer Service dipende esclusivamente dalle interfacce, senza conoscere il formato di persistenza sottostante.

Il principale vantaggio di questo approccio è che il formato di persistenza può essere sostituito, ad esempio passando da JSON a un database relazionale, senza modificare il Model o il Service. Il pattern migliora inoltre la testabilità, poiché le implementazioni concrete dei DAO possono essere sostituite da mock nei test unitari.

2.4 Modello di threading

La simulazione della riproduzione richiede un meccanismo che avanzi il tempo trascorso e gestisca la transizione fra tracce. Il `PlaybackService` utilizza una `Timeline` di JavaFX. A ogni tick (1 secondo) la `Timeline` aggiorna la property `elapsed` e, al termine di una traccia, interroga la `PlaybackStrategy` corrente per determinare la traccia successiva.

2.5 Diagramma delle classi

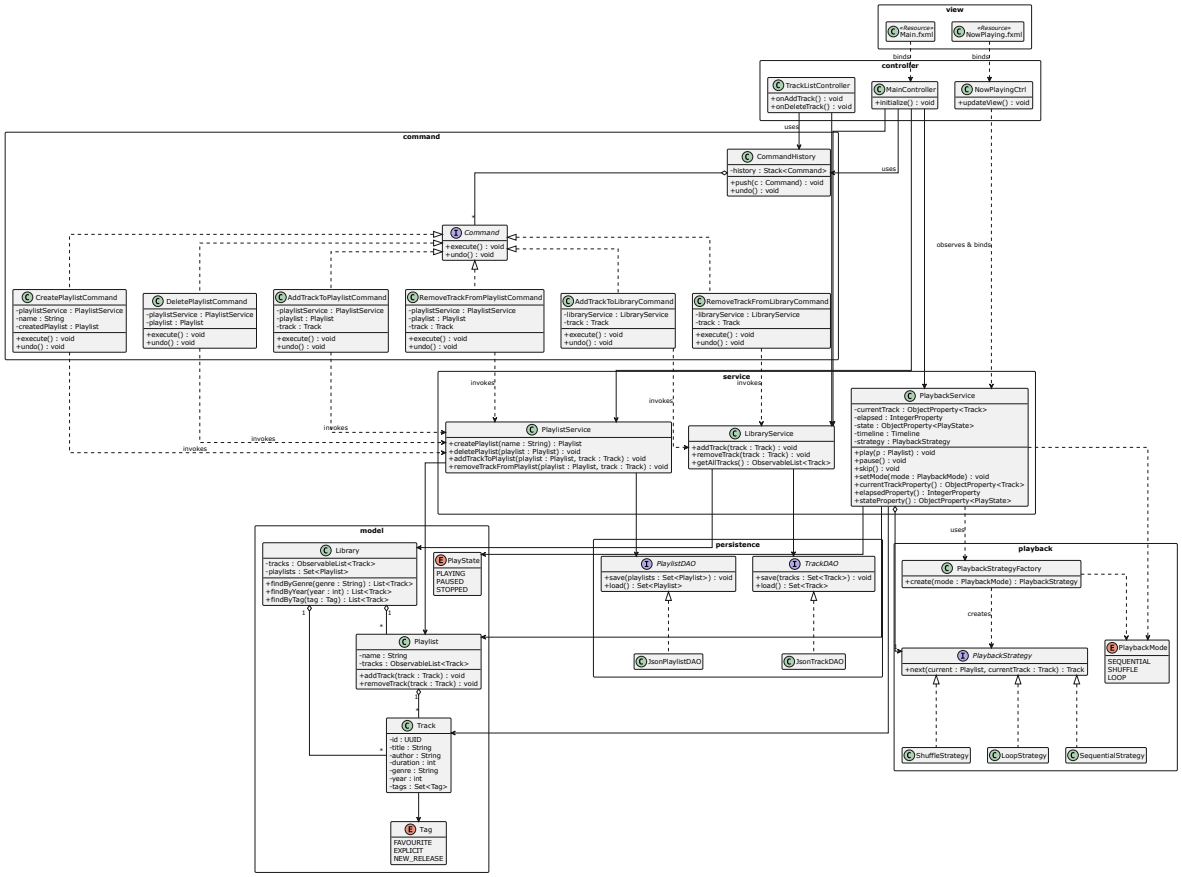


Figure 2: Diagramma delle classi

2.5.1 Lettura del diagramma

Il diagramma rappresenta l'intera architettura statica dell'applicazione, organizzata nei sette package descritti in sezione 2.2. Lo si può percorrere dall'alto verso il basso, seguendo il flusso del controllo che parte dall'input dell'utente e si propaga fino al Model.

Strato di presentazione (view, controller). In cima si trova il package `view`, che contiene esclusivamente risorse FXML (`Main.fxml`, `NowPlaying.fxml`). Queste descrivono la struttura statica dell'interfaccia e sono associate ai controller del package sottostante. Il package `controller` contiene le

tre classi che gestiscono l'interazione utente: `MainController`, responsabile dell'inizializzazione complessiva e del coordinamento tra le viste; `TrackListController`, che gestisce le operazioni su tracce e playlist; `NowPlayingCtrl`, controller del pannello "Now Playing" che mostra la traccia attualmente in riproduzione.

Strato di logica applicativa (service). I controller non contengono logica di business: la delegano al package `service`, che è il cuore dell'applicazione. Esso include `LibraryService` (gestione della libreria), `PlaylistService` (gestione delle playlist) e `PlaybackService` (simulazione della riproduzione). I servizi sono gli unici componenti autorizzati a modificare lo stato del Model.

Strato di dominio (model). Il package `model` raccoglie le entità di dominio: `Track` (titolo, autore, durata, genere, anno, e l'insieme dei tag associati), `Playlist` (nome e lista ordinata di tracce) e `Library` (collezione globale di tracce e playlist, con metodi di filtraggio per genere, anno e tag). I tre enum `Tag`, `PlayState` e `PlaybackMode` rappresentano rispettivamente le etichette visive applicabili a una traccia, lo stato della riproduzione (`PLAYING`, `PAUSED`, `STOPPED`) e la modalità di riproduzione (`SEQUENTIAL`, `SHUFFLE`, `LOOP`).

Package di supporto (playback, command, persistence). Ai lati del livello `service` sono collocati i tre package che ne supportano il funzionamento:

- Il package `playback` contiene l'interfaccia `PlaybackStrategy` e le sue tre implementazioni concrete (`SequentialStrategy`, `ShuffleStrategy`, `LoopStrategy`), insieme alla factory `PlaybackStrategyFactory`. Il `PlaybackService` utilizza una di queste strategie alla volta per decidere quale traccia riprodurre dopo quella corrente.
- Il package `command` contiene l'interfaccia `Command`, la classe `CommandHistory` e le sei classi concrete che incapsulano le operazioni reversibili (aggiunta/rimozione di tracce in libreria e nelle playlist, creazione/eliminazione di playlist). Ogni comando concreto detiene un riferimento al servizio che eseguirà l'azione e ai parametri necessari, oltre allo stato richiesto per il ripristino in caso di undo.
- Il package `persistence` contiene le interfacce `TrackDAO` e `PlaylistDAO` e le rispettive implementazioni `JsonTrackDAO` e `JsonPlaylistDAO`. I servizi vedono soltanto le interfacce; le implementazioni concrete vengono fornite ai servizi al momento della loro creazione.

Tipi di relazione. Le relazioni rappresentate sul diagramma sono di tre tipi:

- **Aggregazione** (linea con cardinalità "1 – *"): rappresenta il contenimento strutturale. Ad esempio, una `Library` contiene un insieme di `Playlist` e di `Track`, e una `Playlist` contiene una lista ordinata di `Track`.
- **Uso/dipendenza** (linea tratteggiata, etichette *uses*, *invokes*, *creates*): rappresenta una dipendenza, tipicamente realizzata tramite parametri di metodo, variabili locali o riferimenti a interfaccia. Ad esempio il `PlaybackService` *usa* la `PlaybackStrategy` corrente, i controller *usano* i servizi e i comandi *invocano* i servizi.
- **Binding/osservazione** (frecce etichettate *binds*, *observes & binds*): indica il canale attraverso cui la view è collegata alle property dei servizi tramite il meccanismo di binding di JavaFX. È il percorso attraverso cui le modifiche di stato si propagano alla UI senza chiamate esplicite da parte dei controller.

3 Flusso del sistema

Questa sezione descrive come i componenti illustrati nelle sezioni precedenti interagiscono a runtime, attraverso una serie di scenari rappresentativi. L'obiettivo è rendere esplicito il percorso che ogni azione utente compie attraverso gli strati dell'architettura, dall'evento di input fino all'aggiornamento dell'interfaccia.

3.1 Avvio dell'applicazione

All'avvio, l'applicazione istanzia i DAO concreti (`JsonTrackDAO`, `JsonPlaylistDAO`) e i tre servizi (`LibraryService`, `PlaylistService`, `PlaybackService`), fornendo ai servizi i riferimenti alle interfacce DAO di cui hanno bisogno. I servizi caricano poi lo stato persistente (libreria e playlist) interrogando i rispettivi DAO, ricostruendo così la situazione lasciata al termine della sessione precedente.
Successivamente `Main.fxml` viene caricato tramite `FXMLLoader`: questo istanzia `MainController`, che riceve i riferimenti ai servizi e predispone i binding tra i componenti grafici e le property esposte dai servizi. A questo punto la libreria e le playlist sono visibili nell'interfaccia, pronte all'interazione.

3.2 Aggiunta di una traccia alla libreria

L'utente seleziona "Aggiungi traccia": il click viene catturato da `TrackListController.onAddTrack()`, che apre un dialog di inserimento (anch'esso definito in FXML). Una volta confermati i dati, il controller costruisce un oggetto `Track` e lo incapsula in un'istanza di `AddTrackToLibraryCommand`, che viene affidata alla `CommandHistory`.
La `CommandHistory` esegue il comando tramite `execute()` e ne conserva un riferimento sullo stack delle operazioni annullabili. Il comando, al suo interno, delega l'effettiva aggiunta a `LibraryService.addTrack()`. Il servizio modifica la `ObservableList<Track>` interna alla `Library`: questa modifica viene automaticamente notificata, tramite il meccanismo di binding di JavaFX, alla `TableView` della libreria, che si aggiorna senza intervento esplicito da parte del controller.
Lo stesso schema (controller → Command → CommandHistory → Service → Model → aggiornamento automatico della UI) si applica a tutte le altre operazioni modificanti: eliminazione e modifica di tracce, creazione e cancellazione di playlist, aggiunta e rimozione di tracce dalle playlist.

3.3 Riproduzione di una playlist

L'utente seleziona una playlist e preme "Play". Il controller corrispondente invoca `PlaybackService.play(playlist)`. Il servizio memorizza un riferimento alla playlist, imposta la property `currentTrack` alla prima traccia, porta la property `state` a `PLAYING` e avvia la `Timeline` di JavaFX.
A ogni tick della Timeline (1 secondo), il servizio incrementa la property `elapsed`. Tutti i componenti grafici legati a questa property — la label che mostra il tempo trascorso, la `ProgressBar` di avanzamento, l'evidenziazione della riga corrente nella `ListView` della playlist — si aggiornano automaticamente attraverso il binding, senza alcuna chiamata esplicita da parte del controller.
Quando `elapsed` raggiunge la durata della traccia corrente, il servizio interroga la `PlaybackStrategy` attualmente attiva tramite `strategy.next(playlist, currentTrack)`, che restituisce la prossima traccia da riprodurre (o un valore che indica la fine della riproduzione, nel caso della strategia sequenziale). La property `currentTrack` viene aggiornata di conseguenza e il pannello "Now Playing", legato a tale property, si aggiorna immediatamente con titolo, autore e contatore del tempo.

3.4 Cambio di modalità di riproduzione a runtime

L'utente seleziona una nuova modalità (ad esempio "Shuffle") da un combobox dell'interfaccia. Il controller invoca `PlaybackService.setMode(PlaybackMode.SHUFFLE)`. Il servizio richiede a `PlaybackStrategyFactory.create(SHUFFLE)` un'istanza di `ShuffleStrategy` e la sostituisce alla strategia corrente.
La traccia attualmente in riproduzione continua finché la Timeline non ne segnala il termine: il cambio di modalità non interrompe l'ascolto. La nuova strategia viene utilizzata soltanto a partire dalla

determinazione della traccia successiva. Questo soddisfa il requisito di poter cambiare modalità "anche durante la riproduzione" senza interruzioni percepibili dall'utente.

3.5 Modifica di una playlist durante la riproduzione

L'utente aggiunge una traccia a una playlist mentre questa è in riproduzione. Il controller invoca `PlaylistService.addTrackToPlaylist(playlist, track)` (sempre incapsulato in un `AddTrackToPlaylistCommand`). Il servizio modifica la lista di tracce della playlist; la `ListView` legata a tale lista si aggiorna istantaneamente.
Poiché il `PlaybackService` detiene un riferimento alla stessa istanza di `Playlist`, la lista aggiornata sarà visibile alla successiva invocazione di `strategy.next()`: la traccia attualmente in riproduzione non subisce interruzioni, poiché la Timeline opera indipendentemente dalla composizione della playlist. Se invece la traccia rimossa coincide con quella attualmente in riproduzione, il comportamento osservabile è quello specificato dal criterio di accettazione della user story US-09 (scenario 2): la riproduzione della traccia eliminata si interrompe e il sistema avanza automaticamente alla traccia successiva determinata dalla strategia corrente.

3.6 Annullamento di un'operazione (undo)

L'utente preme "Undo". Il controller invoca `CommandHistory.undo()`: la classe estrae dallo stack l'ultimo comando eseguito e ne invoca il metodo `undo()`, implementato in modo specifico da ciascuna classe concreta. Ad esempio, `AddTrackToPlaylistCommand.undo()` rimuove la traccia precedentemente aggiunta, mentre `RemoveTrackFromPlaylistCommand.undo()` la reinserisce.
In assenza di operazioni nello stack, `CommandHistory.undo()` non produce effetti, in linea con il criterio di accettazione della user story US-12 che richiede di disabilitare l'operazione di undo quando non vi sono azioni annullabili.

4 Definition of Done

La Definition of Done è l'insieme di criteri condivisi dal team che definiscono quando una user story può essere considerata completata. La sua adozione garantisce uniformità di valutazione tra i membri, evita ambiguità in sede di Sprint Review e protegge il progetto dall'accumulo di debito tecnico nascosto sotto l'apparenza di funzionalità "quasi pronte".
Una User Story si considera "Done" esclusivamente quando soddisfa tutti i seguenti criteri:

4.1 Criteri tecnici sul codice

- **Codice compilabile ed eseguibile.** Il codice sorgente rispetta il pattern architetturale MVC e compila correttamente tramite Maven senza errori né warning bloccanti.
- **Convenzioni di codifica rispettate.** Il codice segue le convenzioni di naming, formattazione e organizzazione (nomi in PascalCase/camelCase, indentazione a 4 spazi, assenza di magic number).
- **Codice documentato.** Le classi e i metodi sono commentati inline al fine di descrivere lo scopo, i parametri e i valori di ritorno.

4.2 Criteri di testabilità e qualità

- **Logica di business indipendente dalla UI.** La funzionalità implementata nei layer Model e Service è testabile senza dover avviare l'interfaccia grafica JavaFX. Questo garantisce che la logica applicativa rimanga disaccoppiata dalla presentazione.
- **Test unitari implementati e passanti.** Sono presenti test JUnit 5 automatizzati che coprono la parte pubblica delle classi di business logic introdotte.
- **Verifica funzionale completata.** La user story è stata provata manualmente sull'interfaccia grafica per accertarsi che il comportamento osservabile dall'utente sia coerente con quanto previsto dai criteri di accettazione.

4.3 Criteri di processo

- **Code review effettuata.** Il codice è stato esaminato da almeno un altro membro del team che ha verificato la qualità del design, la leggibilità e l'aderenza alle convenzioni. Eventuali osservazioni sono state risolte prima del merge.
- **Lavoro integrato.** Il codice sviluppato dal singolo membro, dopo essere stato sottoposto a un rapido code review da parte di un altro componente del team, è stato integrato con successo al codice principale dell'applicazione sulla repository GitHub (tramite merge) senza presentare conflitti.
- **Trello aggiornato.** I task associati alla user story sono stati marcati come completati. La user story è stata spostata nella colonna "Done" della board del progetto.